

Самостійна робота №1: Інструменти збірки та конфігурація проєктів

Тема: Інструменти збірки та конфігурація проєктів

Мета: Ознайомитися з сучасними інструментами збірки JavaScript-проєктів, навчитися конфігурувати Vite, налаштовувати лінер та форматер коду.

Кількість годин: 3

Теоретичний матеріал

Інструменти збірки (Bundlers)

Сучасні веб-застосунки складаються з багатьох модулів, файлів стилів, зображень та інших ресурсів. Інструменти збірки (bundlers) об'єднують ці ресурси у оптимізовані файли для браузера.

Webpack - найпопулярніший bundler з великою екосистемою плагінів. Використовує концепцію loaders для обробки різних типів файлів та plugins для розширення функціональності. Конфігурація описується у файлі `webpack.config.js`. Webpack виконує повну збірку проєкту, що може бути повільним для великих застосунків.

Vite - сучасний інструмент збірки, створений автором Vue.js. Використовує нативні ES-модулі браузера під час розробки, що забезпечує миттєвий старт dev-сервера. Для продакшн-збірки використовує Rollup. Конфігурація описується у `vite.config.ts`. Підтримує React, Vue, Svelte та інші фреймворки "з коробки".

esbuild - надзвичайно швидкий bundler, написаний на Go. Використовується як основа для інших інструментів (наприклад, Vite використовує esbuild для трансформації TypeScript). Підходить для бібліотек та простих проєктів.

Конфігурація Vite

Створення проєкту: `npm create vite@latest my-app -- --template react-ts`. Основні параметри конфігурації: `root`, `base`, `plugins`, `server` (порт, проху), `build` (`outDir`, `sourcemap`). Vite автоматично підтримує TypeScript, JSX, CSS Modules, PostCSS.

ESLint

ESLint - інструмент статичного аналізу коду JavaScript/TypeScript. Знаходить потенційні помилки та забезпечує дотримання стилю коду. Конфігурація описується у файлі `eslint.config.js` (flat config формат). Ключові концепції: `rules` (правила), `plugins` (розширення для фреймворків), `extends` (набори правил).

Prettier

Prettier - автоматичний форматор коду. На відміну від ESLint, який аналізує логіку, Prettier форматує лише зовнішній вигляд коду: відступи, лапки, крапки з комою тощо. Конфігурація у `.prettierrc`. Інтегрується з ESLint через `eslint-config-prettier`.

Контрольні питання

1. У чому основна відмінність підходу Vite від Webpack під час розробки (dev mode)?
2. Яку роль виконує esbuild у екосистемі Vite?
3. Що таке Hot Module Replacement (HMR) і як він працює у Vite?
4. Як налаштувати проху у Vite для перенаправлення API-запитів на backend-сервер?
5. Яка різниця між ESLint та Prettier? Чому їх використовують разом?
6. Що таке flat config у ESLint і чим він відрізняється від класичного `.eslintrc`?
7. Як налаштувати ESLint для роботи з TypeScript та React?

Практичне завдання

1. Створіть новий проєкт за допомогою Vite з шаблоном `react-ts`.
2. Налаштуйте ESLint з правилами для TypeScript та React.
3. Додайте Prettier та інтегруйте його з ESLint.
4. У файлі `vite.config.ts` додайте проху для API-запитів на `http://localhost:3000/api`.
5. Додайте npm-скрипти: `lint`, `lint:fix`, `format`.

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- Vite - офіційна документація - повний гайд по налаштуванню та використанню Vite
- Webpack - офіційна документація - основні концепції Webpack
- esbuild - офіційний сайт - документація esbuild
- ESLint - Getting Started - початок роботи з ESLint
- Prettier - документація - налаштування та інтеграція Prettier
- Vite vs Webpack (стаття) - порівняння підходів Vite та Webpack